

Fair-Share Scheduling

Scheduling by user share, not only process count

Definition

Fair-share scheduling is a scheduling approach that gives each user or group a promised fraction of the CPU, then schedules that user's processes so the promised shares are respected.

1. Problem with Process-Only Scheduling

Usual Assumption

Many scheduling algorithms treat every runnable process as an independent competitor, without considering which user owns it.

Unfair Result

If user 1 starts nine processes and user 2 starts one process, ordinary round robin gives user 1 nine turns for every one turn given to user 2.

User 1 gets 90%, User 2 gets 10%.

User	Number of Processes	CPU Share with Process-Level Round Robin
User 1	9	$9/10 = 90\%$
User 2	1	$1/10 = 10\%$

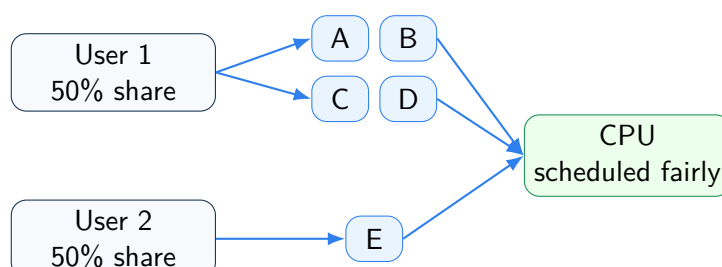
2. Fair-Share Idea

User-Level Fairness

Fair-share scheduling first decides how much CPU each user should receive. The scheduler then chooses among processes in a way that keeps each user close to that promised share.

Main Difference

The unit of fairness is no longer just the process. The scheduler also considers the owner of each process.



3. Promised CPU Fractions

CPU Share

Each user is allocated a fraction of the CPU. If two users are each promised 50%, each should receive half the CPU time, regardless of how many processes they own.

Key Rule

The number of processes owned by a user should not automatically increase that user's total CPU share.

4. Example Setup

Two Users

Assume two users are both promised 50% of the CPU.

- User 1 owns four processes: A, B, C, and D.
- User 2 owns one process: E.

User	Processes	Promised CPU Share
User 1	A, B, C, D	50% total across all four processes
User 2	E	50% total for process E

5. Equal User Shares

Valid Schedule

A possible sequence that gives each user 50% of the CPU is:

A E B E C E D E A E B E C E D E ...

Why It Works

Every other CPU slot goes to E, so user 2 receives half of the CPU. The remaining half is shared among A, B, C, and D, so user 1 also receives half of the CPU.

A E B E C E D E A E B E

User 1 and User 2 alternate CPU ownership

User	Slots in One Cycle	CPU Share
User 1	A, B, C, D	$4/8 = 50\%$
User 2	E, E, E, E	$4/8 = 50\%$

6. Unequal User Shares

User 1 Gets Twice as Much

If user 1 is entitled to twice as much CPU time as user 2, a possible sequence is:

A B E C D E A B E C D E ...

Ratio

In each six-slot cycle, four slots go to user 1 and two slots go to user 2:

User 1 : User 2 = 4 : 2 = 2 : 1.

A B E C D E A B E C D E

User 1 receives two CPU slots for each one of User 2

User	Slots in One Cycle	CPU Share
User 1	A, B, C, D	$4/6 \approx 66.7\%$
User 2	E, E	$2/6 \approx 33.3\%$

7. How the Scheduler Thinks

Two-Level Decision

The scheduler can first decide which user should receive the next CPU slot, then choose one of that user's runnable processes.

choose the user or group most behind its promised CPU share

within that user or group:

choose one runnable process

run it for one time slice

update CPU usage for both the process and its owner

Process Selection Inside a User

Within a user's own share, the system may use round robin, priority scheduling, or another local rule. Fair-share scheduling mainly controls how much total CPU each owner receives.

8. Why This Prevents Abuse

Without Fair Share

A user can get more CPU time simply by starting more processes.

With Fair Share

Starting more processes divides that user's own allocation among more processes. It does not increase the user's total CPU allocation.

9. Different Notions of Fairness

Flexible Policy

Many valid schedules can satisfy the same user-level CPU shares. The exact sequence can vary depending on how the system defines fairness.

Fairness Choice	Meaning
Equal users	Each user receives the same CPU fraction.
Weighted users	Some users receive larger CPU fractions than others.
Group fairness	CPU share is assigned to groups, projects, or departments.
Per-user local policy	Each user's own processes are scheduled within that user's allocation.

10. Comparison with Process-Level Round Robin

Approach	What Is Treated Fairly	Weakness
Process-level round robin	Individual processes.	Users with more processes get more total CPU.
Fair-share scheduling	Users, groups, or owners.	Requires accounting by owner and a share policy.

11. Big Picture

Main Conclusion

Fair-share scheduling prevents CPU share from depending only on how many processes a user starts. It enforces fairness at the owner level by giving each user or group a promised CPU fraction.

Exam Shortcut

Remember: round robin is fair to processes; fair-share scheduling is fair to users or groups. If user 1 owns four processes and user 2 owns one process, fair-share scheduling can still give both users 50% of the CPU.

12. Quick Review

Key Points

- Process-only scheduling ignores who owns each process.
- With ordinary round robin, a user with more processes can receive more total CPU time.
- Fair-share scheduling considers the owner of each process.
- Each user or group is assigned a promised CPU fraction.
- The scheduler chooses processes so the owner-level shares are enforced.
- If two users are promised 50% each, both receive half the CPU even if one owns more processes.
- A valid equal-share sequence is A E B E C E D E
- If user 1 deserves twice as much CPU as user 2, a valid sequence is A B E C D E
- The exact sequence can vary; the important constraint is the CPU-share promise.
- Fair-share scheduling prevents users from gaining CPU share merely by creating more processes.